

Modelling and Parallelized Simulation of Natural Phenomena

N-Body Gravitational Simulation

Submitted by

Pawan Kumar Meena

Roll Number: 24B0904

July 9, 2026

Contents

1	Introduction to the N-Body Problem	2
1.1	The N-Body problem	2
1.2	Historical Background	2
2	Special Cases with Analytical Solutions	3
2.1	The Two-Body Problem	3
2.2	Euler's Collinear Solutions	5
2.3	Lagrange's Triangular Solutions	6
2.4	Restricted Three-Body Problem	6
3	Numerical methods	6
3.1	Core Idea	6
3.2	Time Integration Methods	6
3.2.1	Taylor Expansion	7
3.2.2	Off-the-shelf Integrators	7
3.2.3	Symplectic Integrators	8
3.3	Force Calculation Methods	9
3.3.1	Direct Summation	9
3.3.2	Barnes-Hut Algorithm	9
4	Design and Implementation on CPU	10
4.1	Design	10
4.1.1	Particles and Vectors	10
4.1.2	Force Calculation	10
4.1.3	Time Integration	10
4.1.4	Timing statistics	10
4.1.5	Directory Structure	11
4.2	Implementation	11
4.3	Understanding the Barnes-Hut Algorithm	12
4.4	Building the Tree	12
4.5	Force Calculation	13
4.6	Optimization	14
5	Implementation on GPU	16
5.1	Design	16
5.1.1	Philosophy behind a good CUDA implementation	17
5.2	Algorithm Implementation of GPUs	17
5.3	Barnes-Hut Algorithm on GPU	17
5.3.1	Parallel Tree Construction	18
5.3.2	Tree Destruction	19
5.4	Optimizations	20
5.5	Performance	20

1 Introduction to the N-Body Problem

1.1 The N-Body problem

The N-body problem is a fundamental problem in physics that involves predicting the future state(i.e. position and velocity) of each component in a system of N celestial bodies interacting with each other through gravitational forces. Now, this might seem like a straightforward problem, and much unrelated to computing, but that is very much not the case.

1.2 Historical Background

This first began in the 17th century, when Sir Issac Newton published his work "*Philosophiæ Naturalis Principia Mathematica*" in 1687 which laid the foundation for classical mechanics and provided the mathematical framework to describe the gravitational force between two bodies.

$$F = G \frac{m_1 m_2}{r^2} \quad (1)$$

Where:

- F is the gravitational force between the two bodies.
- G is the gravitational constant.
- m_1 and m_2 are the masses of the two bodies.
- r is the distance between the centers of the two bodies.

In theory, these equations could be used to turn the unpredictability of the universe to a deterministic system. Given a initial starting state of the objects, we can use the newton's gravitational formula, the laws of motion and the knowledge of calculus to predict the future state of the system at any time, in the future or in the past. However, in practical, we only have the exact solution(also known as analytic solution) for just the two-body problem. Adding even one more body to the system adds chaos to the motion of the bodies and makes it impossible to solve the equations analytically.

The N-body problem first came up when newton tried to apply the analytical two-body solution to the motion of planets in the solar system, after collecting the initial states. But over the course of a few years, he and others soon discovered that the motion of the planets was not exactly as predicted by the two-body solution. They realized that this was because gravitational interactive forces amongst all the planets were affecting all their orbits.

Since, they were able to solve the two-body problem easily, with the likes of kepler already having derived similar results(like the elliptical orbit) even before the formulation of gravity, the obvious next step was to solve the three boyd problem and the N-body problem. However, even after three centuries of research, there has been no practical analytical solution to the N-body problem other than a few specialized cases or conditions.

In the late 1800s, mathematicians Henri Poincaré and Ernst Burns asserted that no general solution exists for the N-body problem, and that the dynamics of the system is inherently chaotic, and depends sensitively on the initial conditions. This means that even

a small error in the initial state of the system can lead to vastly different outcomes over time, making long-term predictions extremely difficult. This was also one of the earliest discoveries of chaos theory, and a significant blow to the deterministic or the predictable view of the universe.

2 Special Cases with Analytical Solutions

While we do not have a practical analytical solution to the N-body problem, there has been several remarkable progress in the field with exact solutions for specialized cases, research about the analytical solutions and the development of numerical methods to approximate the solution. There are a few special cases of the N-body problem that have been solved analytically.

2.1 The Two-Body Problem

The two-body problem is the simplest non-trivial case of the N-body problem, where only two bodies are interacting with each other. This problem can be solved analytically using the law of universal gravitation, and langrangian mechanics.

Consider two point masses, m_1 and m_2 , located at positions $\mathbf{r}_1$ and $\mathbf{r}_2$, respectively. Let $\mathbf{r} = \mathbf{r}_2 - \mathbf{r}_1$ be the relative position vector from m_1 to m_2 . Then the force equation would be:

$$\mathbf{F} = -G \frac{m_1 m_2}{|\mathbf{r}|^3} \mathbf{r} \quad (2)$$

Now, take the Langrangian of the system, which is the difference between the kinetic energy and the potential energy:

$$L = T - U \quad (3)$$

Where:

- T is the kinetic energy of the system
- U is the potential energy of the system

Now let's try to calculate the kinetic energy of the system.

$$T = \frac{1}{2} m_1 |\dot{\mathbf{r}}_1|^2 + \frac{1}{2} m_2 |\dot{\mathbf{r}}_2|^2 \quad (4)$$

The problem is that we have two position vectors, giving us six degrees of freedom. So we need to reduce them. Let $\mathbf{R}$ be the center of mass of the system. Then:

$$\mathbf{R} = \frac{m_1 \mathbf{r}_1 + m_2 \mathbf{r}_2}{M} \quad (5)$$

Where M is the total mass of the system and we know:

$$\mathbf{r} = \mathbf{r}_2 - \mathbf{r}_1 \quad (6)$$

Using (5) and (6), we can express $\mathbf{r}_1$ and $\mathbf{r}_2$ in terms of $\mathbf{R}$ and $\mathbf{r}$:

$$\mathbf{r}_1 = \mathbf{R} - \frac{m_2}{M} \mathbf{r} \quad (7)$$

$$\mathbf{r}_2 = \mathbf{R} + \frac{m_1}{M} \mathbf{r} \quad (8)$$

Substituting these into the expression for kinetic energy, we get:

$$T = \frac{1}{2} M |\dot{\mathbf{R}}|^2 + \frac{1}{2} \mu |\dot{\mathbf{r}}|^2 \quad (9)$$

Where μ is the reduced mass of the system, defined as:

$$\mu = \frac{m_1 m_2}{M} \quad (10)$$

Now take the Langrangian of the system using (3) substituting the expressions for kinetic energy:

$$L = \frac{1}{2} M |\dot{\mathbf{R}}|^2 + \frac{1}{2} \mu |\dot{\mathbf{r}}|^2 + U(r) \quad (11)$$

Now, we can apply the Euler-Lagrange equations to derive the equations of motion for the system. The Euler-Lagrange equations are given by:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = 0 \quad (12)$$

Where q_i are the generalized coordinates of the system. In our case, we can choose $\mathbf{R}$ as the generalized coordinates. Applying the Euler-Lagrange equations to $\mathbf{R}_x$ gives:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{R}_x} \right) - \frac{\partial L}{\partial R_x} = 0 \quad (13)$$

Now if we notice, L doesn't depend on $\mathbf{R}$, but only on $\dot{\mathbf{R}}$. So the second term of the above equation is zero. So we get:

$$\frac{d}{dt} (M \dot{R}_x) = 0 \quad (14)$$

This implies that $M \dot{R}_x$ is a constant, which means that the center of mass of the system moves with a constant velocity. So we can now choose the frame of reference of the center of mass, which means that $\dot{\mathbf{R}} = 0$, as we can later just add the constant velocity of the center of mass to the solution to get the solution in any frame of reference and similarly take $\mathbf{R}$ as zero too.

Now lets look at the angular momentum of the system. The angular momentum $\mathbf{L}$ is given by:

$$\mathbf{L} = \mathbf{r}_1 \times m_1 \dot{\mathbf{r}}_1 + \mathbf{r}_2 \times m_2 \dot{\mathbf{r}}_2 \quad (15)$$

Substituting the expressions for $\mathbf{r}_1$ and $\mathbf{r}_2$ in terms of $\mathbf{R}$ and $\mathbf{r}$, we get:

$$\mathbf{L} = \mu \mathbf{r} \times \dot{\mathbf{r}} \quad (16)$$

Now the angular momentum is conserved, which means that $\mathbf{L}$ is a constant. This implies that the motion of the two bodies is confined to a plane perpendicular to the angular momentum vector (One can show this easily by taking the time derivative of the angular momentum and showing that it is zero).

So let's take the plane of motion as the xy-plane, meaning that $\mathbf{r}$ can be written as $\mathbf{r} = (x, y, 0)$. So we are only left with two degrees of freedom. Now convert this to the

polar coordinates, where r is the distance between the two bodies and θ is the angle between the line connecting the two bodies and the x-axis. So we can write:

$$x = r \cos \theta \quad (17)$$

$$y = r \sin \theta \quad (18)$$

We can now express the kinetic energy in terms of r and θ :

$$T = \frac{1}{2}\mu(\dot{r}^2 + r^2\dot{\theta}^2) \quad (19)$$

Write the langrangian again, now in terms of r and θ :

$$L = \frac{1}{2}\mu(\dot{r}^2 + r^2\dot{\theta}^2) + U(r) \quad (20)$$

Now we can apply the Euler-Lagrange equations to derive the equations of motion for r and θ . Applying the Euler-Lagrange equations to θ gives:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right) - \frac{\partial L}{\partial \theta} = 0 \quad (21)$$

Now if we notice, L doesn't depend on θ , but only on $\dot{\theta}$. So the second term of the above equation is zero. So we get:

$$\frac{d}{dt} (\mu r^2 \dot{\theta}) = 0 \quad (22)$$

This implies that $\mu r^2 \dot{\theta}$ is a constant. We can denote this constant as l .

$$\mu r^2 \dot{\theta} = l \quad (23)$$

Now applying the Euler-Lagrange equations to r gives:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{r}} \right) - \frac{\partial L}{\partial r} = 0 \quad (24)$$

Substituting the expression for L , we get:

$$\frac{d}{dt} (\mu \dot{r}) - \left(\mu r \dot{\theta}^2 - \frac{dU}{dr} \right) = 0 \quad (25)$$

Substituting the expression for $\dot{\theta}$, we get:

$$\mu \ddot{r} - \frac{l^2}{\mu r^3} + \frac{dU}{dr} = 0 \quad (26)$$

Thus we got an equation with only one degree of freedom, which can be solved analytically to get the solution for r as a function of time.

2.2 Euler's Collinear Solutions

In 1767, Leonhard Euler discovered a special class of solutions to the three-body problem in which the three bodies remain on a single straight line at all times. The ratios of the distances between the bodies remain constant, preserving the shape of the configuration as it moves. These solutions are known as Euler's collinear solutions.

2.3 Lagrange's Triangular Solutions

Discovered by Joseph-Louis Lagrange in 1772, these solutions describe a configuration where three bodies form an equilateral triangle that rotates around the center of mass. The distances between the bodies remain constant, and the triangle maintains its shape as it moves through space. These solutions are known as Lagrange's triangular solutions.

2.4 Restricted Three-Body Problem

The restricted three-body problem is a special case of the three-body problem where one of the bodies has negligible mass compared to the other two. This allows for an analytical solution to be derived for the motion of the negligible mass body under the gravitational influence of the two larger bodies. This problem is often used to study the motion of satellites and spacecraft in the vicinity of two larger celestial bodies, such as the Earth and the Moon. While still being an approximation, it provides valuable insights into the dynamics of such systems and has practical applications in space exploration.

3 Numerical methods

Untill now, we only have the analytical solution to a few special cases of the N-body problem. For the general case, we have no practical solution to getting the accurate future state of these systems. Hence, these problems have to solved or more precisely simulated using numerical methods.

Numerical methods are techniques used to approximate the solutions of mathematical problems that cannot be solved, which can be either becuae we cannot solve the procedure analytically or because the analytical solution is intractable.

3.1 Core Idea

Solving the N-body problem numerically is basically a two-step process.

1. Calculate the forces acting on each body due to the gravitational attraction of all other bodies in the system.
2. Update the positions and velocities of each body based on the calculated forces using numerical integration.

Now depending upon the needs of the simulation, like the accuracy, computation resources available, the number of bodies in the system, and the time duration of the simulation, we have different methods to solve these steps.

Let's first look at the time integration methods, which are used to update the positions and velocities of the bodies based on the calculated forces.

3.2 Time Integration Methods

Firstly let's try to understand the problem and what we are trying to solve and find.

For a system of N bodies, let's say the position and velocity of the i^{th} body at time t are given by $\mathbf{r}_i^t$ and $\mathbf{v}_i^t$ respectively. The force acting on the i^{th} body due to the gravitational

attraction of all other bodies can be calculated using Newton's law of universal gravitation as:

$$\mathbf{F}_i^t = G \sum_{j \neq i} \frac{m_i m_j}{|\mathbf{r}_i^t - \mathbf{r}_j^t|^3} (\mathbf{r}_j^t - \mathbf{r}_i^t) \quad (27)$$

Where G is the gravitational constant, and m_i and m_j are the masses of the i^{th} and j^{th} bodies respectively. Rewriting it into differential form, we get:

$$\frac{d\mathbf{v}_i}{dt} = \frac{\mathbf{F}_i}{m_i} \quad (28)$$

$$\frac{d\mathbf{r}_i}{dt} = \mathbf{v}_i \quad (29)$$

Now we want to find the position and velocity of each body at a future time T . But since we don't know the exact solution to these equations, we try to move towards T in small time steps, say Δt , and update the positions and velocities of the bodies at each time step. Hence this process is called time integration.

3.2.1 Taylor Expansion

Now for calculating position (and similarly velocity) at a future time $t + \Delta t$, we can use the Taylor expansion of the position function $\mathbf{r}_i(t)$ around time t :

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \Delta t \frac{d\mathbf{r}_i}{dt} + \frac{(\Delta t)^2}{2!} \frac{d^2\mathbf{r}_i}{dt^2} + \frac{(\Delta t)^3}{3!} \frac{d^3\mathbf{r}_i}{dt^3} + \dots \quad (30)$$

Now, we can truncate this series after a certain number of terms to get an approximation of the position at time $t + \Delta t$. The more terms we include, the more accurate our approximation will be, but it will also require more computational resources. Truncating to the **n-th** term gives us an approximation of order n , known as the **n-th order method**.

Using this Taylor series, we can derive various time integration methods. These methods aren't just specifically used for the N-body problem, but are general methods used for solving ordinary differential equations and are very commonly used as off the shelf methods for solving such differential equations in various fields of science and engineering.

3.2.2 Off-the-shelf Integrators

There are various off-the-shelf integrators available for solving ordinary differential equations, which can be used for the N-body problem as well.

1. **Euler's Method:** This is the simplest numerical integration method, which uses the first-order Taylor expansion and approximates the curve as a straight line between two points.

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \Delta t \mathbf{v}_i(t) \quad (31)$$

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \Delta t \mathbf{a}_i(t) \quad (32)$$

2. **Runge-Kutta Methods:** These are a family of higher-order methods that provide better accuracy by evaluating the function at multiple points within the time step.

- (a) **RK2**: Second order method, also known as the midpoint method.

$$k_1 = f(t, y) \quad (33)$$

$$k_2 = f\left(t + \frac{\Delta t}{2}, y + \frac{\Delta t}{2}k_1\right) \quad (34)$$

$$y(t + \Delta t) = y(t) + \Delta t k_2 \quad (35)$$

- (b) **RK4**: Fourth order method, which is the most commonly used Runge-Kutta method.

$$k_1 = f(t, y) \quad (36)$$

$$k_2 = f\left(t + \frac{\Delta t}{2}, y + \frac{\Delta t}{2}k_1\right) \quad (37)$$

$$k_3 = f\left(t + \frac{\Delta t}{2}, y + \frac{\Delta t}{2}k_2\right) \quad (38)$$

$$k_4 = f(t + \Delta t, y + \Delta t k_3) \quad (39)$$

$$y(t + \Delta t) = y(t) + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (40)$$

While these methods already provide a good accuracy, they have a major disadvantage at the fact these equations do not conserve energy, as they are only trying to approximate the curve, without considering the underlying physics of the problem. This can lead to significant errors in long-term simulations, as the energy of the system can drift over time, leading to unphysical results.

3.2.3 Symplectic Integrators

Symplectic integrators are a class of numerical integration methods that are designed to preserve the phase-space structure of Hamiltonian systems, which includes the N-body problem. These integrators are particularly useful for long-term simulations of conservative systems, as they help to maintain the energy and other conserved quantities of the system over time. Symplectic integrators are based on the idea of splitting the Hamiltonian into separate parts, which can be integrated exactly or approximately, and then combining the results to obtain an overall update for the system.

Symplectic Integrators in itself is a specialized topic and is beyond the scope of this report and project and we'll be directly using some of the most commonly used symplectic integrators, the symplectic Euler, the Leapfrog and the Velocity Verlet integrators, which are both second-order methods.

1. **Symplectic Euler Integrator**: In normal Euler method, we first update the position using the old velocity, and then update the velocity using the old acceleration. This means that the position is updated without considering the inward acceleration which can lead to loss or increase in energy of the system. An easy way to fix this is to first update the velocity using the old acceleration, and then update the position using the new velocity. This is the symplectic Euler method.

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \Delta t \mathbf{a}_i(t) \quad (41)$$

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \Delta t \mathbf{v}_i(t + \Delta t) \quad (42)$$

2. Leapfrog Integrator:

$$\mathbf{v}_i(t + \frac{\Delta t}{2}) = \mathbf{v}_i(t) + \frac{\Delta t}{2} \mathbf{a}_i(t) \quad (43)$$

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \Delta t \mathbf{v}_i(t + \frac{\Delta t}{2}) \quad (44)$$

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t + \frac{\Delta t}{2}) + \frac{\Delta t}{2} \mathbf{a}_i(t + \Delta t) \quad (45)$$

3. Velocity Verlet Integrator:

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \mathbf{v}_i(t) \Delta t + \frac{1}{2} \mathbf{a}_i(t) \Delta t^2 \quad (46)$$

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \frac{1}{2} (\mathbf{a}_i(t) + \mathbf{a}_i(t + \Delta t)) \Delta t \quad (47)$$

3.3 Force Calculation Methods

The other step in the numerical simulation of the N-body problem is to calculate the forces acting on each body due to the gravitational attraction of all other bodies in the system. Now this can be done in many ways and is commonly the bottleneck in the simulation, as it requires calculating the forces between all pairs of bodies, which can be computationally expensive for large systems and brings in the diversity of methods to calculate the forces.

3.3.1 Direct Summation

The most straightforward method to calculate the forces is to directly sum the contributions from all other bodies for each body. This method has a computational complexity of $O(N^2)$, as we need to calculate the force between each pair of bodies. We can optimize this by using the fact that the force between two bodies is equal and opposite, so we only need to calculate the force for each pair once. However, this method is still computationally expensive for large systems, and is not practical for simulating systems with a large number of bodies.

3.3.2 Barnes-Hut Algorithm

The Barnes-Hut algorithm is a hierarchical method for approximating the forces in an N-body simulation. It works by dividing the simulation space into a hierarchical tree structure, where each node of the tree represents a region of space containing a group of bodies. The algorithm then approximates the forces from distant nodes by treating them as a single body located at the center of mass of the node. This allows for a significant reduction in the number of force calculations, as distant nodes can be approximated with fewer calculations. The Barnes-Hut algorithm has a computational complexity of $O(N \log N)$, making it much more efficient than direct summation for large systems.

4 Design and Implementation on CPU

4.1 Design

A good and efficient implementation of a computationally extensive problem like the N-body simulation requires a good design of the data structures and algorithms used in the implementation. The design should be modular, allowing for easy modification and extension of the code. We have to also consider the performance of the implementation, using efficient data structures and algorithms, having an idea of the memory access patterns and cache usage, and minimizing the long latency memory accesses. The design should also consider the parallelization of the code, allowing for efficient use of multiple CPU cores.

I had initially implemented the N-body simulation with the only goal being correctness and focusing on the quality of the code, and not the performance. This was done to ensure that the implementation was correct and to have a good understanding of the problem and the solution. Once I was confident that the implementation was correct, I then focused on optimizing the performance of the code.

I implemented a modular design, with separate classes for the different components of the simulation:

4.1.1 Particles and Vectors

The particles are represented as a class, with the position, velocity, acceleration, mass and Id(name) of the particle as the attributes, with appropriate constructors.

Since we are dealing with a lot of vectors, I also implemented a vector class with different vector operations like addition, subtraction, dot product, cross product, normalization and also overloaded the operators for these operations along with some other utility functions like printing the vector, getting the magnitude of the vector and getting the unit vector. The vector class is used for the position, velocity and acceleration of the particles.

4.1.2 Force Calculation

A base class for force calculation is implemented which serves as an interface for the different force calculation methods. The base class has a virtual function for calculating the acceleration of the particles, which is overridden by the derived classes for the different force calculation methods. The derived classes implement the direct summation and Barnes-Hut algorithm for force calculation.

4.1.3 Time Integration

A base class for time integration is implemented which serves as an interface for the different time integration methods. The base class has a virtual function step, which is overridden by the derived classes for the different time integration methods.

4.1.4 Timing statistics

For analysing the timing and performance of the implementation, I implemented a timer class which uses clock from the chrono library to measure the time taken for

different parts of the simulation. The timer class has functions to record time, reset the timer and get the elapsed time. The timer class is used to measure the time taken for force calculation, time integration and the total time taken for the simulation and is a major component of the performance analysis of the implementation and finding the bottlenecks in the implementation.

4.1.5 Directory Structure

The CPU implementation has been divided into 3 different directories, 1 directory for the outputs

```
CPU
├── include
│   ├── common.hpp
│   ├── particle.hpp
│   ├── force_calculator.hpp
│   ├── integrator.hpp
│   └── test_common.hpp
├── src
├── tests
├── outputs
└── Makefile
```

- **include:** This directory contains the header files for the different classes and functions used in the implementation. The header files contain the class definitions and function declarations, along with the necessary includes and using directives. It also contains header files of all the derived classes.
- **src:** This directory contains the source files for the different classes and functions used in the implementation. The source files contain the class definitions and function implementations, along with the necessary includes and using directives.
- **tests:** This directory contains the test files for testing different components of the implementation.
- **outputs:** This directory contains the output files generated by the simulation. The output files will usually contain the outputs of simulations, comparison against other methods, timing statistics of different parts of the simulation and errors in the simulation.
- **Makefile:** This file is used for compiling the code quickly, without needing to type the whole command. It contains the command to run the code on test cases or the main simulator.

4.2 Implementation

The implementation of the N-body simulation is done in C++, using the classes and design described above. While the implementation of most are simple and straightforward, the implementation of the Barnes-Hut algorithm is a bit more complex and requires a good understanding of the algorithm and the data structures used in the implementation.

4.3 Understanding the Barnes-Hut Algorithm

The Barnes-Hut algorithm is a way of approximating the forces based on their distance and the width of the region they occupy

$$\frac{s}{d} < \theta \quad (48)$$

Where s is the width of the region, d is the distance from the center of mass of the region to the particle and θ is a threshold parameter that controls the accuracy of the approximation. If the ratio $\frac{s}{d}$ is less than θ , then we can approximate the forces from all the particles in that region as a single force from the center of mass of that region.

We first divide the space into 8 regions and then for particle check the value of the ratio $\frac{s}{d}$ for each region. If the ratio is less than θ , then we can approximate the forces from all the particles in that region as a single force from the center of mass of that region. But if the ratio is bigger? We subdivide those regions into 8 subregions and then repeat this process.

4.4 Building the Tree

The first step in the Barnes-Hut algorithm is to build the tree structure that represents the spatial hierarchy of the particles. The tree is built recursively, starting from the root.

Node: A node will store the information of each Node. It contains the information regarding the mass of node, size of node, center of the node, center of mass of the node, the particles in the node and the child nodes of the node.

- **Root Node:** The root node of the tree represents the entire simulation space, and contains all the particles in the simulation. We first calculate the bounding box of the simulation by finding the extremes of all three axes and find the minimal cube that contains all the particles. This cube is the root node of the tree.
- **Recursive Insertion:** Now we insert each particle into the tree one by one. For each particle, we start at the root node and check if the node is a leaf node or an internal node. Now a leaf node is a node that doesn't have children. While an internal node is a node that has children nodes. So if the region in which the particle belong to, is a leaf, then we check the leaf capacity. Leaf capacity is the maximum number of particles that can be stored in a leaf node. If the leaf capacity is not exceeded, then we add the particle to the leaf node. But if the leaf capacity is exceeded, then we subdivide the leaf node into 8 child nodes and redistribute the particles among the child nodes based on their positions. We then repeat this process for each child node until all particles are inserted into the tree.
- **Updating Node Information:** After inserting a particle into the tree, we update the information of the node, including the mass, center of mass, and size of the node. The mass of the node is the sum of the masses of all particles in the node, the center of mass is the weighted average of the positions of all particles in the node, and the size of the node is the length of the side of the cube that contains all particles in the node.

While the above process is simple and straightforward, it can be a bit tricky to implement correctly, especially when dealing with edge cases like particles that are very close to each other or how to avoid using dynamic memory allocation or a dynamic container.

Edge case: One issue that one can face is the case when more than leaf capacity number of particles are extremely close. What will happen is that they will try to occupy the same node, but the leaf capacity can't allow this, so the node will continue to subdivide infinitely. To prevent this, we can set a minimum size for the node, and if the node is smaller than this size, then we can just add the particle to the node, even if it exceeds the leaf capacity. This will prevent the tree from subdividing infinitely and will also prevent the tree from becoming too deep. But this will cause another problem. This means we cannot use a fixed sized container to store the particles in the node, as we don't know how many particles will be in the node. So we have to end up using a dynamic container like a vector to store the particles in the node. Why this is bad is because unlike array, vector has a lot of overhead and is not very cache friendly. I end up using vectors for each node which is one of the bottleneck.

Destroying the Tree: After each time step, we have to destroy as the tree no longer correct information due to particle's changed positions. So we have to destroy the tree, before rebuilding it for the next time step. This can be done recursively, starting from the root node and deleting all child nodes before deleting the parent node.

Performance: After a correct and decent attempt at implementing, the performance were this:

Table 1: Performance metrics for tree operations

Number of Particles	Building Tree(in ms)	Destroying Tree(in ms)
100	0.0326974	0.0283557
500	0.146484	0.134858
1,000	0.317474	0.305257
5,000	1.65714	1.30534
10,000	3.61331	3.05595
100,000	69.8663	53.9589

So the performance of the tree operations looks good, with the trees building and destroying times being in a few milliseconds for a few thousand particles, but the performance drops significantly as we scale up to larger number particles. building and destroying tree for 100K particles, alone stopped any chance of simulation being done in real time, with them combining to take more than 120ms.

4.5 Force Calculation

After building the tree, we can calculate the forces acting on each particle due to the gravitational attraction of all other particles in the system. This is done by traversing the tree and calculating the forces from each node based on the distance and size of the node. If the ratio $\frac{s}{d}$ is less than θ , then we can approximate the forces from all the particles in that node as a single force from the center of mass of that node. But if the ratio is bigger, we have to traverse the child nodes of that node and calculate the forces from each child node.

This is fairly simple and straight forward to implement with no major issues in implementation. Recursively traversing the tree and calculating the forces from each node based on the distance and size of the node. To avoid recursive function calls, we can a stack-based iterative approach to traverse the tree, which is also a bit more efficient than the recursive approach.

So in the end our overall performance looks like this:

Table 2: Average time per operation for Barnes-Hut (ms)

N	Building Tree	Traversal	Destroying Tree	Total Avg time
100	0.0326974	0.158799	0.0283557	0.213912
500	0.146484	2.23927	0.134858	2.45561
1,000	0.317474	5.48634	0.305257	6.09221
5,000	1.65714	48.9468	1.40074	51.6998
10,000	3.61331	158.070	3.20986	165.967
100,000	69.8663	5738.75	53.9589	5883.49

*Note: Total time can be checked in the log entries; slight variance exists in each run.

So each time step takes more than 5 seconds for 100K particles, which is not good enough for real-time simulation. So we have to optimize the implementation further to get better performance. The biggest bottleneck in the implementation is easily tree traversal which alone takes more than 95% of the time for 100K particles, while also being the major component of every case.

4.6 Optimization

The initial implementation of the Barnes–Hut algorithm produced correct results but was still considerably slower than expected. Although the theoretical complexity of Barnes–Hut is approximately $O(N \log N)$, constant factors such as dynamic memory allocation, poor cache utilization, recursive function calls, and pointer indirection resulted in poor practical performance. Profiling revealed that tree traversal accounted for more than 95% of the execution time, while tree construction and destruction also introduced significant overhead. Several optimizations were therefore incorporated to improve both memory efficiency and computational performance.

Preallocated Node Pool: The original implementation dynamically allocated every tree node using `new` and recursively freed the nodes using `delete` after every timestep. Dynamic allocation is relatively expensive because each allocation requires interaction with the memory allocator and can lead to memory fragmentation.

To eliminate this overhead, the optimized implementation allocates a large contiguous pool of nodes once during initialization. Whenever a new node is required during tree construction, it is simply obtained by incrementing an index into this pool. Since the entire pool is reused every timestep, destroying the tree no longer requires recursively deleting every node; instead, rebuilding the tree simply resets the allocation index to zero. This optimization almost completely removes the cost of tree destruction while also improving memory locality during tree construction.

Contiguous Child Storage: In the original implementation, every node stored eight separate child pointers. Since each child could be allocated independently, the children

of a node were often scattered throughout memory, resulting in poor cache utilization during traversal.

The optimized implementation stores only the index of the first child. Because all eight children are allocated consecutively from the node pool, the remaining child indices are simply consecutive locations in memory. This guarantees that sibling nodes are stored contiguously, improving cache locality and reducing memory accesses during tree traversal.

Index-Based Tree Representation Rather than storing pointers between nodes, the optimized implementation represents the tree entirely using integer indices into the node pool.

Using indices instead of pointers reduces the memory footprint of every node and avoids unnecessary pointer dereferencing during traversal. Since all nodes reside inside one contiguous array, the processor cache is utilized much more effectively, resulting in better traversal performance.

Optimizing Leaf Capacity: The leaf capacity determines the maximum number of particles stored in a leaf node before subdivision occurs. A very small leaf capacity produces a deeper tree with many approximation nodes, while a very large leaf capacity reduces the depth of the tree but increases the amount of direct particle-particle computation performed inside each leaf.

Multiple leaf capacities were experimentally evaluated for different particle counts. It was observed that no single value performs best for all problem sizes. For smaller systems, a larger leaf capacity reduces unnecessary subdivision, whereas larger simulations benefit from smaller leaf capacities that produce more effective spatial partitioning. Based on benchmarking, a leaf capacity of 32 performs well for moderate particle counts, while a value of 8 provides the best performance for larger simulations.

Parallel Force Calculation: Once the octree has been constructed, the acceleration of every particle can be computed independently because the tree is only read during traversal. This makes the force evaluation stage highly parallel.

The optimized implementation therefore parallelizes the outer particle loop using OpenMP:

```
#pragma omp parallel for schedule(static)
```

Each thread computes the acceleration of a different particle while all threads share the same read-only octree. Since no synchronization is required during traversal, this optimization provides a significant reduction in execution time on multi-core processors.

Iterative Tree Traversal: The original implementation traversed the octree recursively. Although recursion simplifies implementation, every recursive call introduces additional function call overhead and increases stack usage.

The optimized implementation replaces recursion with an explicit stack-based iterative traversal. This removes recursive function overhead, improves branch prediction, and avoids deep call stacks for highly subdivided trees. While this optimization alone provides only a modest improvement, its impact becomes significant because traversal is executed once for every particle.

Caching Frequently Used Quantities: Several quantities used repeatedly during force computation remain constant for each node, such as the square of the node size and the square of the opening angle threshold. Instead of recomputing these values during every traversal, they are precomputed once and stored inside the node.

Similarly, the gravitational softening parameter is stored in squared form. This reduces the number of floating-point multiplications performed inside the innermost force calculation loop, where even small savings become significant due to the large number of interactions evaluated.

The combined effect of these optimizations substantially improved the overall performance of the Barnes-Hut implementation. Tree construction became approximately two to four times faster, tree destruction became effectively negligible due to the reusable node pool, and traversal benefited from improved cache locality, reduced recursion overhead, and multi-threaded execution. Overall, the optimized implementation achieved a speedup of approximately $14\times$ for simulations containing 100 000 particles while maintaining similar force accuracy to the original implementation.

Table 3: Average time per operation and speedup for Barnes-Hut Optimized (ms)

N	Building	Traversal	Destroying	Total Avg	Speedup
100	0.0157515	0.0553516	0.00003718	0.062379	3.42924x
500	0.0928900	0.4629810	0.00004470	0.548240	4.47909x
1,000	0.2178900	1.0877300	0.00005226	1.303830	4.67255x
5,000	0.9528680	7.3572100	0.00004858	8.315630	6.21718x
10,000	1.9829500	15.421000	0.00005055	17.38600	9.546x
50,000	13.4747	103.305	0.0005591	116.988	NA
100,000	19.881500	403.28900	0.00005007	427.7900	13.7532x

*Note: Speedup is against the original barnes hut implementation.

By the end of these optimizations, we can easily simulate 10K particles and also 50k particles with around 8 steps per second(8 fps), which is a good enough performance for real-time simulation for CPUs. But we have hit the limit of what we can do with the CPU implementation, with any sort of optimization or parallelization, we cannot go beyond 100K particles with a good enough performance for real-time simulation. Hence we have to move to the GPU implementation to get better performance and be able to simulate more particles in real-time.

5 Implementation on GPU

The GPU implementation of the N-body simulation is done using CUDA, which is a parallel computing platform and programming model developed by NVIDIA for general computing on GPUs. CUDA allows developers to write programs that can run on the GPU, taking advantage of its massive parallel processing capabilities.

5.1 Design

I have tried to keep the design of the GPU implementation as similar as possible to the CPU version. The biggest change in the implementation is the philosophy behind the implementation.

5.1.1 Philosophy behind a good CUDA implementation

When we implemented the CPU version, we have structs for Particles, vectors, Nodes, timing and for a lot of other things. But this is inherently a flawed way to work in CUDA. The design we followed in CPU version is known as **Array of Structs(AoS)**. Meaning we have array of particles, where each element is a struct containing the position, velocity, acceleration, mass and Id of the particle. While being fine for CPU, this is a major bottleneck for GPUs.

GPUs rely on thousand of cores with small amount of memory and a lot of threads running in parallel. So to get the best performance out of the GPU, we have to make sure that the memory accesses are coalesced, meaning that the threads should access the memory in a contiguous manner. This is not possible with AoS, as each thread will be accessing different parts of the struct, leading to uncoalesced memory accesses and poor performance. A lot of information in AOS is also not required by each thread, leading to a waste of an already small memory.

Hence, we change our design philosophy to **Structure of Arrays(SoA)**. Meaning we have separate arrays for position, velocity, acceleration, mass and everything. There's no struct or anything(well, technically we are still using them, but we are not storing arrays of them and only used when the required data is just a single frame of data). Everything is stored in arrays, which are contiguous in nature and fixed in size. This allows for coalesced memory accesses and better performance on the GPU.

ParticleArrays: This struct contains all the arrays for position, velocity, acceleration, mass and everything we will be needing.

5.2 Algorithm Implementation of GPUs

All the integrators and force calculation methods are implemented as CUDA kernels, which are functions that can be executed on the GPU. Naive force implementation and time integrators are simple kernel implementations, while using techniques like shared memory and parallel reduction to optimize the performance.

5.3 Barnes-Hut Algorithm on GPU

This is the most complex part of the whole project. While the concept of Barnes-Hut remains the same, the whole algorithm changed. Let's look at the steps of the CPU version of Barnes-Hut:

1. Initialize the root
2. Build the tree by inserting each particle into the tree
3. Update COM and mass of each while insertion of particles in the tree
4. Traverse the tree for each particle to calculate the forces
5. Destroy the tree

Now if we look at the above steps, is building tree parallelizable? Well before that, let's look at the traversal part. Is traversal parallelizable? Well yes, in fact it's embarrassingly parallel, as each particle can be traversed independently. So we can just launch one thread per particle and calculate the acceleration for each particle independently.

Now what about building the tree? Well, in CPU version, what we did was we inserted one particle, which changed the whole tree. And then insert the next particle which again changed the whole tree. This is the main issue. You can't simply parallelize the building, as each part is dependent on the previous part, which breaks parallelization.

This calls for a whole new way of building trees.

5.3.1 Parallel Tree Construction

The sequential insertion-based tree construction used in the CPU implementation is fundamentally unsuitable for GPUs. In the CPU version, particles are inserted into the octree one at a time. Each insertion may split nodes, create new children, redistribute particles, and update the mass and center of mass of every ancestor node. Consequently, the structure of the tree after inserting particle i determines how particle $i + 1$ will be inserted. Since every insertion depends on the result of the previous insertion, the algorithm is inherently sequential.

This dependency prevents us from assigning one thread to each particle during tree construction, since multiple threads would attempt to modify the same nodes simultaneously, leading to race conditions and requiring expensive synchronization. Even if atomic operations were used, the amount of contention would eliminate most of the benefits of GPU parallelism.

Instead of inserting particles one by one, the GPU implementation follows an entirely different approach. Rather than constructing the tree dynamically, we first sort the particles according to their spatial locations and then construct the tree level by level in parallel. Every stage of the construction becomes an independent parallel operation, allowing thousands of GPU threads to participate simultaneously.

Step 1: Normalizing Particle Coordinates: The first step is to determine the bounding box containing all particles. Similar to the CPU implementation, we compute the minimum and maximum coordinates along each axis and construct the smallest cube that contains every particle. Every particle position is then normalized into the interval $[0, 1]$ along each dimension,

$$x' = \frac{x - x_{\min}}{L},$$

where L is the side length of the bounding cube. The same normalization is performed for the y and z coordinates.

This transformation ensures that every particle lies inside the unit cube regardless of the actual scale of the simulation.

Step 2: Converting Positions to Morton Codes: Once normalized, each coordinate is converted into an integer using a fixed number of bits. The three integer coordinates are then interleaved to produce a single Morton code (also known as a Z-order curve).

For example,

$$(x, y, z) \rightarrow (x_2x_1x_0, y_2y_1y_0, z_2z_1z_0)$$

is converted into

$$x_2y_2z_2x_1y_1z_1x_0y_0z_0.$$

This interleaving preserves spatial locality, meaning particles that are physically close together usually receive similar Morton codes. Consequently, sorting particles by Morton code also arranges nearby particles next to one another in memory. Unlike directly sorting three-dimensional coordinates, Morton encoding converts the spatial ordering problem into a simple one dimensional sorting problem while still approximately preserving neighborhood relationships. Since every particle computes its own Morton code independently, this stage is perfectly parallel and launches one CUDA thread per particle.

Step 3: Parallel Radix Sort: After generating the Morton codes, the particles are sorted according to these codes using a parallel radix sort. After sorting, particles belonging to the same spatial region appear consecutively in memory. This ordering is the key observation that makes the remainder of the tree construction possible. Instead of searching the entire particle array to determine which particles belong to a node, every node simply stores a contiguous range of particle indices.

Step 4: Building Internal Nodes: Once the particles are sorted, the tree is constructed from the sorted Morton codes rather than from particle insertions. The longest common prefixes between neighbouring Morton codes determine where particles begin to diverge spatially. These divergence points uniquely determine the parent-child relationships within the octree. Each internal node can therefore be constructed independently by examining only a small neighbourhood of Morton codes rather than interacting with the rest of the tree. Since every internal node depends only on the sorted Morton codes and not on other nodes being constructed, thousands of nodes can be generated simultaneously by GPU threads without synchronization. This completely removes the sequential dependency present in the CPU implementation.

Step 5: Computing Mass and Center of Mass: Once the tree topology has been generated, each leaf node already contains a contiguous range of particles. The mass and center of mass of every leaf are computed directly from these particles. The internal nodes are then processed in reverse order, accumulating the masses and centers of mass of their children.

Although this stage introduces a dependency between parents and children, the dependency exists only between different levels of the tree rather than individual particles. All nodes belonging to the same level can therefore be processed simultaneously. Compared to updating the center of mass during every insertion in the CPU implementation, this bottom-up reduction performs substantially less work while exposing a large amount of parallelism.

5.3.2 Tree Destruction

The tree destruction process is very similar to the optimized CPU implementation. Since the tree is constructed from a preallocated pool of nodes, destroying the tree simply involves resetting the allocation index to zero. This allows the entire node pool to be reused for the next timestep without any dynamic memory deallocation.

5.4 Optimizations

While not much optimizations, there are still a few optimizations that can be done to improve the performance of the GPU implementation.

Sorted arrays: Since we obtain the sorted indices of the particles after sorting them based on their Morton codes, we can use these sorted indices to access the particle data in a more cache-friendly manner. This is because the particles that are spatially close to each other will also be close to each other in memory, which improves cache locality and reduces memory access latency. So the whole traversal uses the sorted indices to access the particle data, which improves the performance of the traversal.

Precision: While we have dealt with **double** datatype this whole project, CUDA is much more optimized in floating arithmetic and calculations. So we store a lot of information in floating point, though carefully. The particle information are still in double, however the whole tree and the logic is written while using float datatypes and casting doubles to floats.

Stack size: Stack used in the iterative traversal has been set to the size as the maximum depth of the stack reached, while simulating $N = 1M$ particles, setting it to 50, with the maximum stack depth being 41. This avoids the stack from living in the local memory(L1 cache) and is mostly on registers, which are much faster, though increasing pressure on the register usage.

5.5 Performance

Table 4: Average time per operation for GPU Barnes-Hut (ms)

N	Building Tree	Traversal	Destroying Tree	Total Avg Time
100	0.295200	0.143204	0.008814	0.447218
500	0.405979	0.357894	0.009573	0.773446
1,000	0.426108	0.405711	0.009975	0.841794
5,000	0.971951	0.829669	0.008892	1.810511
10,000	1.508405	1.088885	0.008391	2.605681
50,000	5.854640	6.358250	0.008584	12.221474
100,000	12.535250	13.025150	0.009607	25.570007
500,000	67.917350	112.416500	0.018445	180.352295
1,000,000	126.836500	246.477500	0.027436	373.342000

*Values are averaged over multiple runs and may changed with different GPUs.

This performance is a huge improvement over the CPU implementation, with the GPU implementation being able to simulate 100K particles in just 25ms. However, simulating one million particles still takes around 373ms, which is not good enough for real-time simulation. I am sure with more optimization we can bring this to less than 50ms and be able to do simulate it in real time, but I have reached the limit of my knowledge and skills in CUDA programming and optimization. Hence I will be leaving it at this.